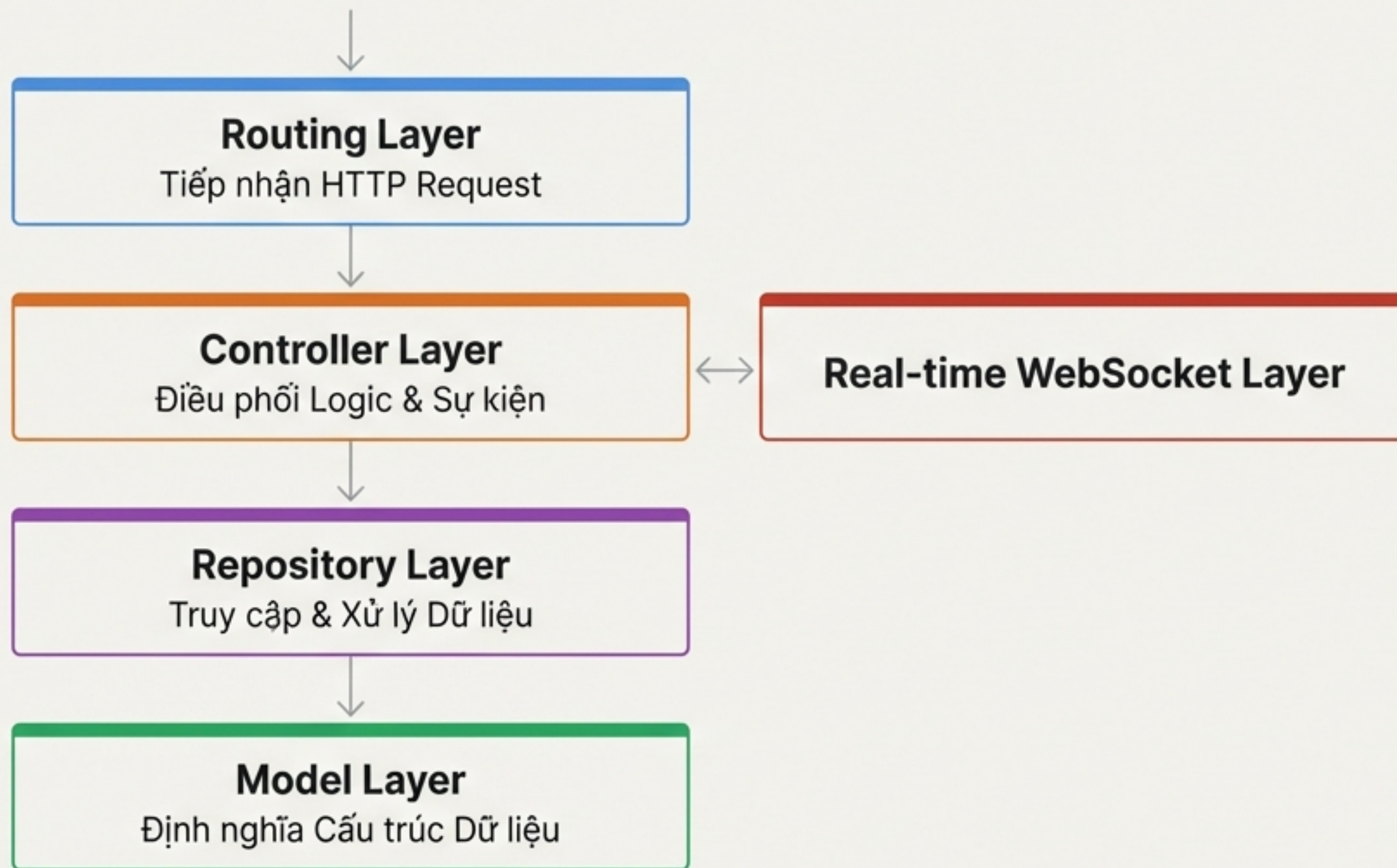


Xây dựng Nền tảng cho Sự kết nối

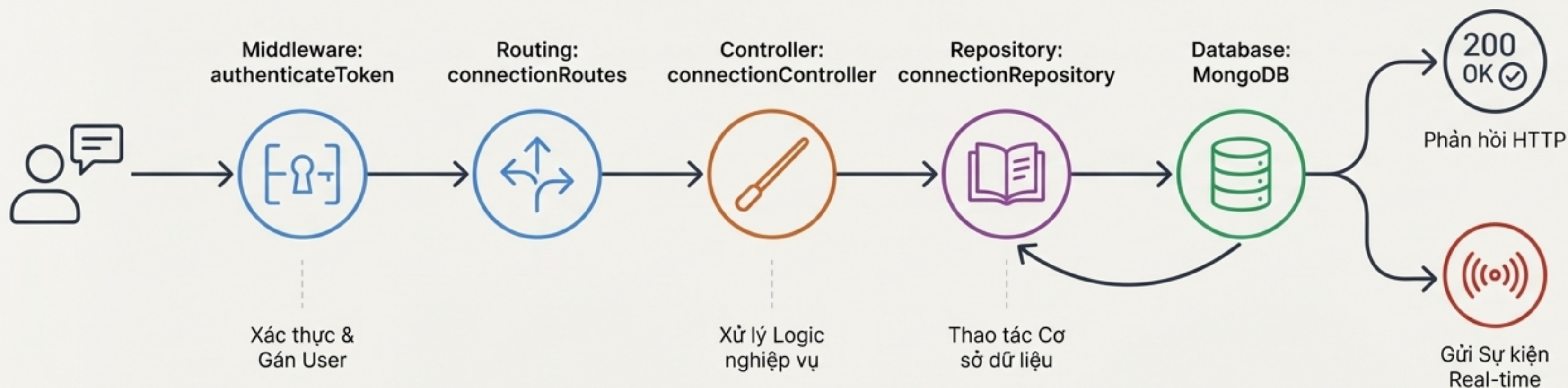
Phân tích Chi tiết Kiến trúc Microservice
Quản lý Bận bè Thời gian thực

Tổng quan Kiến trúc: Một Hệ thống Rõ ràng và Tách biệt



Hệ thống được xây dựng theo kiến trúc **MVC + Repository Pattern**, một lựa chọn có chủ đích để tối ưu hóa khả năng bảo trì, kiểm thử và phân tách các mối quan tâm (separation of concerns).

Hành trình của một Yêu cầu: Luồng Dữ liệu từ Đầu đến Cuối



Lớp Model: Nền tảng Dữ liệu và Nguồn Chân lý Duy nhất

`Connection.js` - Định nghĩa cấu trúc cho mọi mối quan hệ.

```
// Connection.js Schema
const ConnectionSchema = new mongoose.Schema({
  requester: { type: ObjectId, ref: 'User', index: true },
  recipient: { type: ObjectId, ref: 'User', index: true },
  status: {
    type: String,
    enum: ['pending', 'accepted', 'rejected', 'blocked'],
    index: true
  },
  blockedBy: { type: ObjectId, ref: 'User' },
  timestamps: true
});
```



Quyết định Thiết kế Trọng tâm

Sử dụng **Enum** cho trường `status` để thực thi một state machine nghiêm ngặt, đảm bảo dữ liệu luôn ở trạng thái hợp lệ (`pending`, `accepted`, v.v.) và ngăn chặn các trạng thái không mong muốn.

Tối ưu hóa Model: Đảm bảo Hiệu năng và Toàn vẹn Dữ liệu



Chiến lược Đánh chỉ mục (Indexing Strategy)

Compound Index (Unique): {requester: 1, recipient: 1}

Mục đích: Đảm bảo chỉ có một bản ghi kết nối duy nhất giữa hai người dùng, ngăn chặn các lời mời trùng lặp. Tốc độ tra cứu $O(1)$.

Single Indexes: {requester: 1, status: 1} và {recipient: 1, status: 1}

Mục đích: Tăng tốc độ truy vấn danh sách bạn bè, lời mời đang chờ và các bộ lọc theo trạng thái khác.



Đảm bảo Toàn vẹn Dữ liệu (Data Integrity)

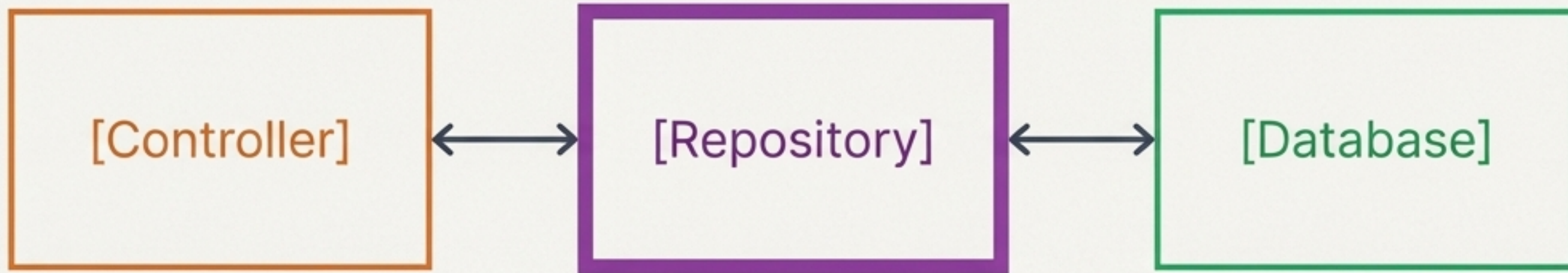
Pre-save Hook

```
// Ngăn người dùng tự kết bạn với chính mình
ConnectionSchema.pre('save', function(next) {
  if (this.requester.equals(this.recipient)) {
    next(new Error('Requester and recipient
cannot be the same person.'));
  }
  next();
});
```

Logic được tích hợp ngay tại tầng Model để tự động ngăn chặn người dùng gửi yêu cầu cho chính mình, đảm bảo tính hợp lệ của dữ liệu ngay từ nguồn.

Lớp Repository: Người Gác cổng cho Mọi Logic Dữ liệu

connectionRepository.js - Trung tâm xử lý nghiệp vụ, đảm bảo tính nhất quán.



- `sendRequest`: Kiểm tra và tạo lời mời mới.
- `acceptRequest` / `rejectRequest`: Quản lý và cập nhật trạng thái lời mời.
- `getFriends` / `getPendingRequests`: Truy xuất danh sách bạn bè và lời mời.
- `blockUser` / `unlockUser`: Xử lý logic chặn/bỏ chặn.
- `getSuggestions`: Thực thi logic phức tạp để gợi ý bạn bè.

Quyết định Thiết kế Trọng tâm

Tập trung toàn bộ logic truy xuất và xử lý dữ liệu vào Repository. Controller sẽ không bao giờ tương tác trực tiếp với Model, giúp mã nguồn trở nên sạch sẽ, dễ kiểm thử và dễ bảo trì.

Bên trong Repository: Logic Vững chắc cho các Tình huống Phức tạp

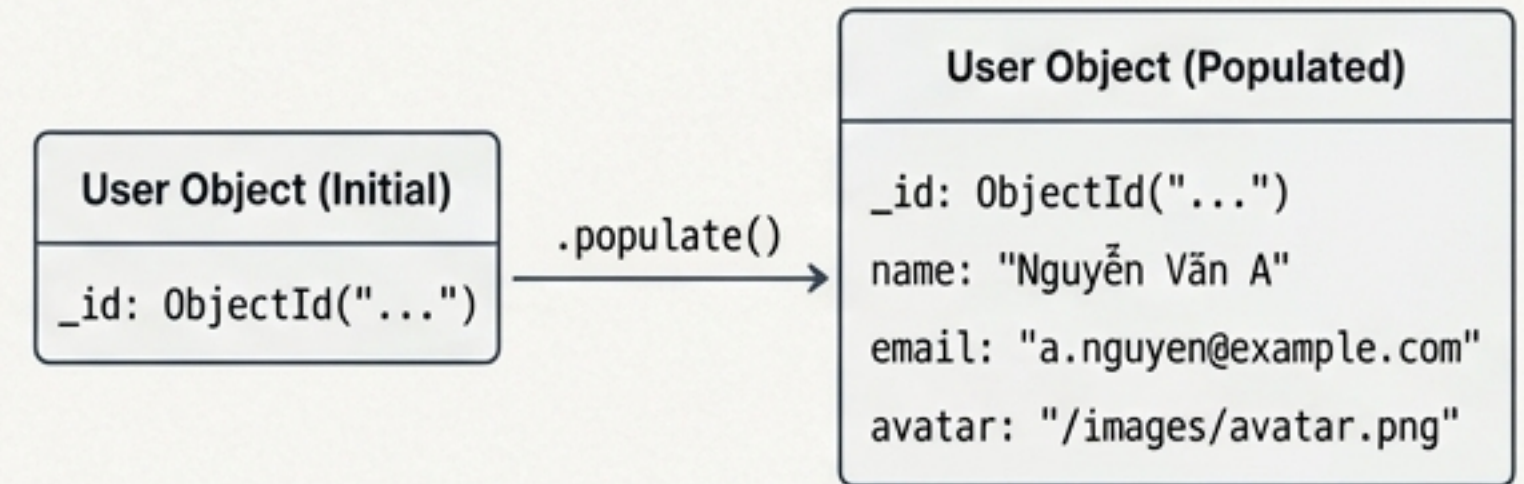
Truy vấn Hai chiều (Bidirectional Queries)

Vấn đề: Một mối quan hệ bạn bè giữa User A và User B có thể được lưu trữ dưới dạng `requester: A, recipient: B` hoặc ngược lại. Làm thế nào để tìm thấy nó một cách hiệu quả?

Giải pháp: Sử dụng toán tử `\$or` của MongoDB để kiểm tra cả hai khả năng trong một truy vấn duy nhất.

```
// Tìm kết nối bất kể ai là người gửi
const existingConnection = await Connection.findOne({
  $or: [
    { requester: userA, recipient: userB },
    { requester: userB, recipient: userA }
  ]
});
```

Chiến lược Population Thông minh



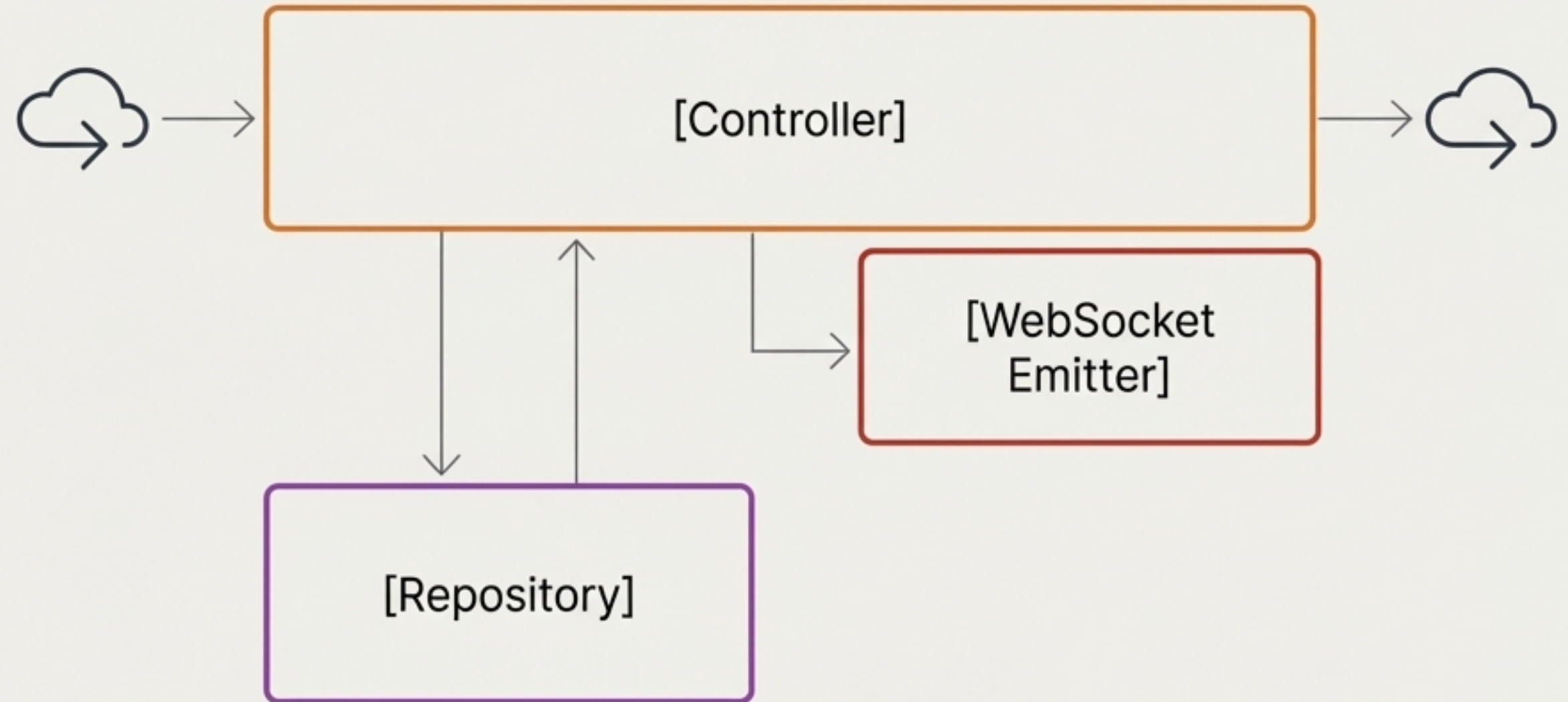
Mọi phương thức trả về dữ liệu người dùng (ví dụ: `getFriends`) đều tự động `populate` các trường cần thiết (tên, email, avatar). Điều này giúp giảm đáng kể số lượng lệnh gọi API từ phía client, cải thiện trải nghiệm người dùng.

Lớp Controller: Cầu nối giữa Yêu cầu HTTP và Hành động Hệ thống

`connectionController.js` - Xử lý request, điều phối logic và phát sóng sự kiện.

Key Responsibilities

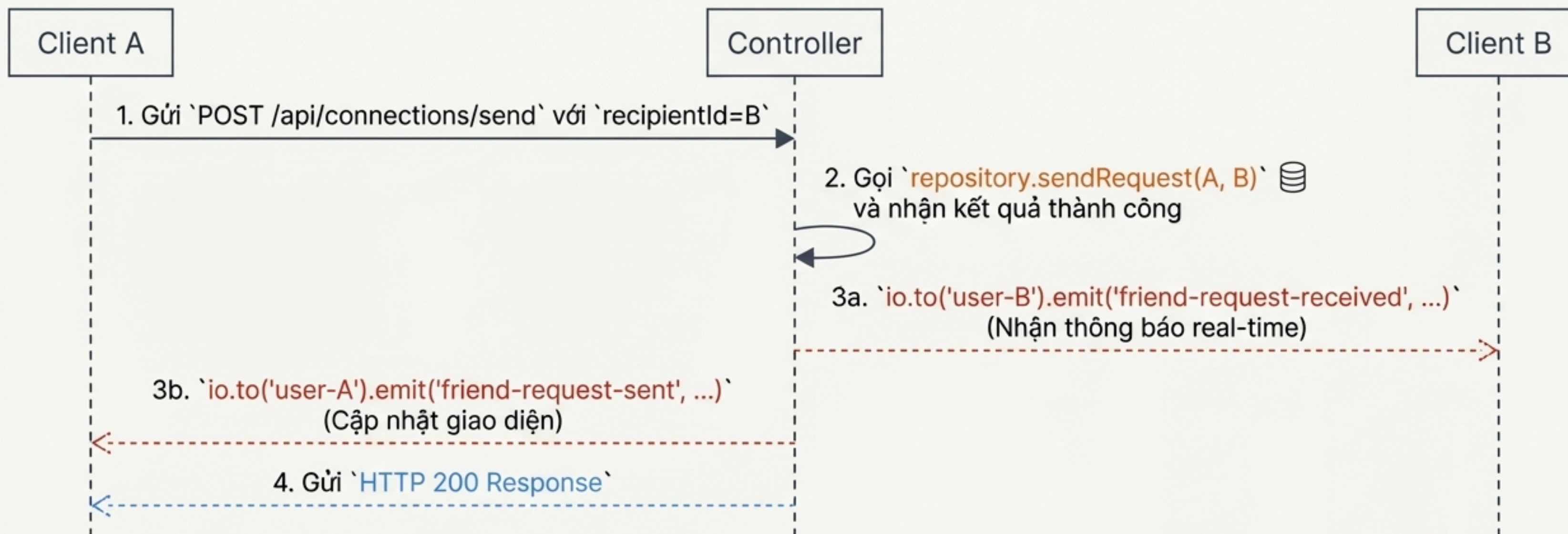
- ✓ **Validation**
Xác thực dữ liệu đầu vào từ request (ví dụ: `recipientId` là bắt buộc).
- 🔧 **Orchestration**
Gọi các phương thức phù hợp từ `connectionRepository`.
- (🔗) **Real-time Integration**
Tích hợp với Socket.IO để phát các sự kiện tới client liên quan.
- 📄 **Response Formatting**
Xây dựng và gửi phản hồi JSON nhất quán cho client.



🛡️ Security Note

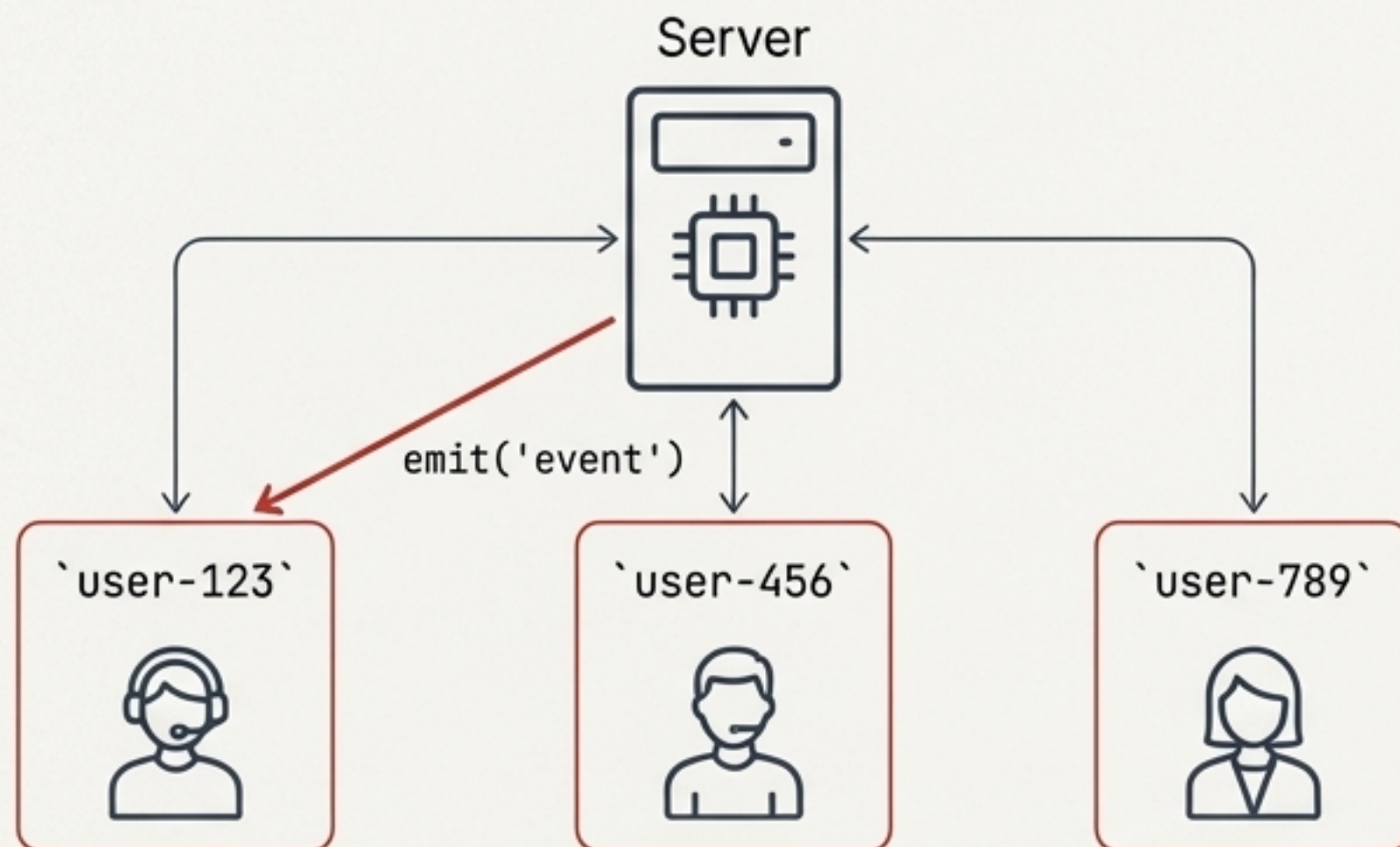
Tất cả các route đều được bảo vệ bởi middleware `authenticateToken`, đảm bảo chỉ những người dùng đã xác thực mới có thể thực hiện hành động.

Case Study: Gửi một Lời mời Kết bạn



Hệ thống không chỉ xử lý hành động mà còn cung cấp phản hồi tức thì cho *tất cả* các bên liên quan. Người gửi nhận được xác nhận, và người nhận nhận được thông báo ngay lập tức, tạo ra một trải nghiệm liền mạch.

Trực xương sống Real-time: Kiến trúc Socket.IO



****Xác thực Socket**:**

Client kết nối kèm JWT token. Middleware

``socketAuth`` xác thực và gán ``socket.userId``.



****Chiến lược Room (Room Strategy)**:**

Mỗi người dùng sau khi kết nối sẽ tự động tham gia vào một room cá nhân (``user-${userId}``).



****Phát sóng có mục tiêu (Targeted Emission)**:**

Thay vì phát sóng toàn cục, các sự kiện được gửi chính xác đến các room của người dùng liên quan. Ví dụ: ``io.to('user-123').emit(...)``.

Quyết định Thiết kế Trọng tâm

Sử dụng **room cá nhân** cho mỗi người dùng là yếu tố then chốt cho khả năng mở rộng. Nó đảm bảo hệ thống không bị quá tải bởi các broadcast không cần thiết và giữ cho việc giao tiếp hiệu quả và an toàn.

Giao diện API: Rõ ràng, nhất quán và theo chuẩn RESTful

`connectionRoutes.js` - Định nghĩa các điểm cuối (endpoints) của hệ thống.

Quản lý Lời mời

POST /send
POST /accept/:requestId
DELETE /cancel/:requestId

Quản lý Bạn bè

GET /friends
DELETE /unfriend/:friendId

Quản lý Chặn

POST /block/:targetUserId
DELETE /unblock/:targetUserId

Truy vấn thông tin

GET /requests
GET /suggestions
GET /status/:targetUserId

Toàn bộ các **route** trong file này đều được áp dụng **middleware** `router.use(authenticateToken)`, đảm bảo một lớp bảo mật đồng nhất trên toàn bộ API.

Ba Trụ cột của một Hệ thống Vững chắc

Tối ưu Hiệu năng (Performance)



Tối ưu Hiệu năng (Performance)

Database Level

Đánh chỉ mục thông minh (Compound & Single indexes).

Application Level

Sử dụng Aggregation Pipeline cho các truy vấn phức tạp, `lean()` queries để giảm overhead.

Population Strategy

Hạn chế API calls từ client.

Bảo mật Toàn diện (Security)



Bảo mật Toàn diện (Security)

Authentication

Yêu cầu JWT token cho mọi API và kết nối Socket.

Authorization

Kiểm tra quyền hạn nghiêm ngặt ở tầng Repository (VD: chỉ người nhận mới được chấp nhận lời mời).

Validation

Ngăn chặn tự kết bạn, chống trùng lặp dữ liệu từ tầng Model và Database.

Tầm nhìn Mở rộng (Scalability)



Tầm nhìn Mở rộng (Scalability)

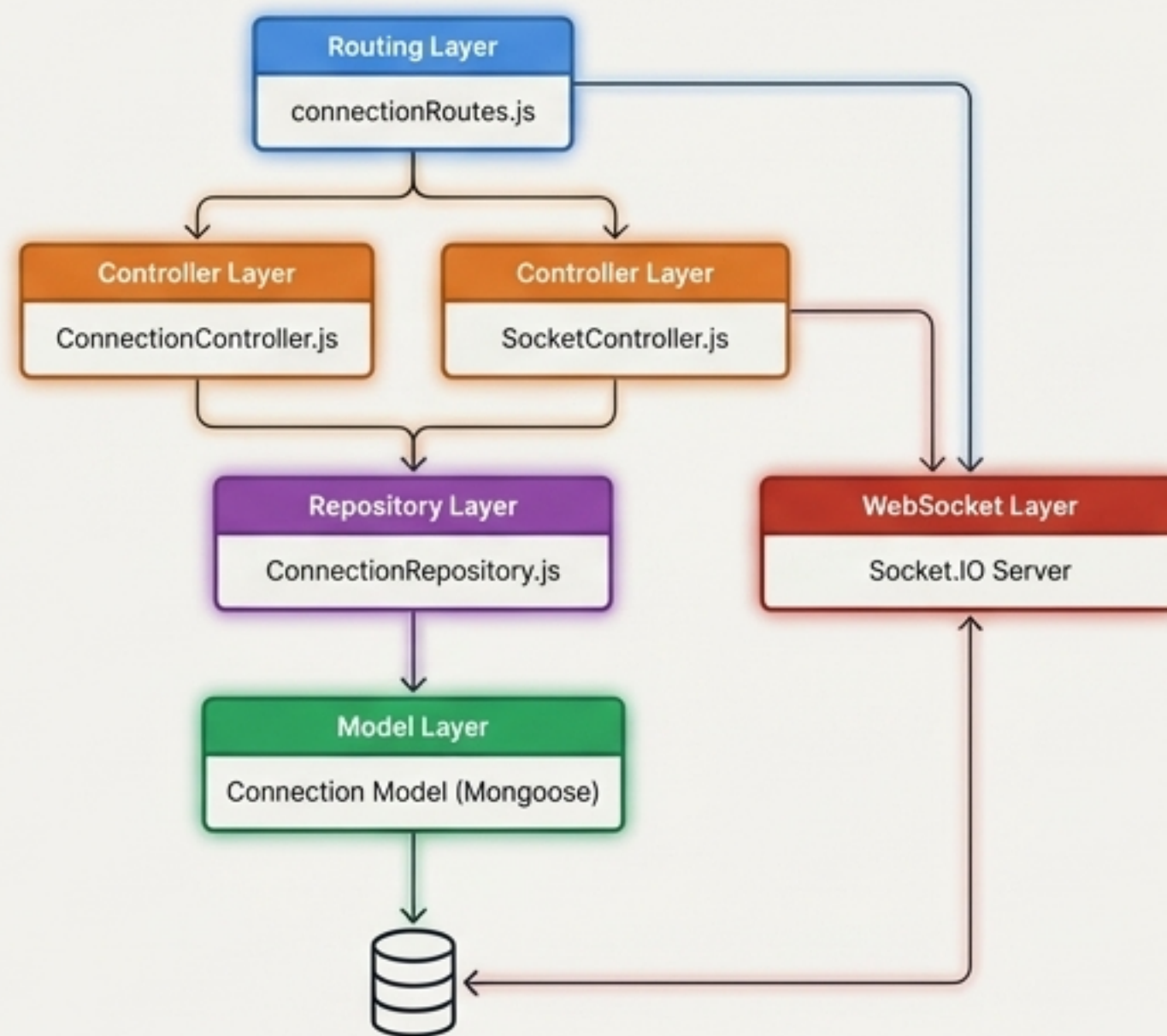
Hạn chế hiện tại

Socket.IO rooms trong bộ nhớ không mở rộng theo chiều ngang. Query gợi ý có thể chậm với dữ liệu lớn.

Hướng cải tiến

Sử dụng Redis Adapter cho Socket.IO, cache kết quả gợi ý, phân trang cho danh sách bạn bè, tính toán gợi ý bằng background job.

Một Kiến trúc Hoàn chỉnh: Mỗi Lớp đều được Thiết kế có Chủ đích



Từ cấu trúc dữ liệu được tối ưu ở tầng Model, logic nghiệp vụ chặt chẽ ở Repository, cho đến sự điều phối linh hoạt của Controller và hệ thống WebSocket tức thời, mỗi thành phần đều được xây dựng để tạo nên một nền tảng kết nối đáng tin cậy, hiệu quả và sẵn sàng cho tương lai.